悪性サーバ検出のための LLMマルチエージェントによるIoTマルウェアの通信解析

篠宮 一真[1,a)]　中山 雄介[1]

概要：IoT マルウェアの C&C サーバやローダサーバなどの悪性サーバを早期に把握することは，DDoS 攻撃や感染拡大を抑止する上で重要である．従来の悪性サーバ探索手法は，観測済み通信やマルウェアの動的実行に依存するため，通信挙動を十分に観測できない場合があり，侵襲性のあるパケット送信のリスクや，判定根拠の説明性不足も課題である．本研究では，LLM エージェントを用いて IoT マルウェア検体に含まれる通信処理を MCP を介して静的解析し，通信挙動の抽出，根拠照合，探査プログラム（Prober）の生成，および安全性審査を行う枠組みを提案する．Mirai や Gafgyt を含む 4 ファミリ 120 検体を用いた評価の結果，TCP 接続が成立した Prober の約 70%で応答一致を確認し，LLM による通信挙動抽出が実通信に基づく悪性サーバ識別に有効であることを示した．さらに，生成した Prober を用いた探索により，入力検体に含まれる既知の通信先に限定されない悪性サーバ候補を発見できることを確認した．一方で，暗号化や復号鍵の導出処理を含む通信への対応には課題が残ることも明らかになった．

LLM Multi-Agent-Based Communication Analysis of IoT Malware for Malicious Server Discovery

Kazuma Shinomiya[1,a)]　Yusuke Nakayama[1]

Abstract: Early identification of malicious servers used by IoT malware, such as C&C servers and loader servers, is important for suppressing DDoS attacks and containing the spread of infection. Existing methods for discovering malicious servers depend on previously observed traffic or on dynamic execution of malware, and therefore cannot always observe communication behavior sufficiently; the risk of intrusive packet transmission and the lack of explainability in the resulting decisions are further problems. In this study, we propose a framework in which an LLM agent statically analyzes, via MCP, the communication routines contained in IoT malware samples, and performs extraction of communication behavior, evidence checking, generation of probing programs (Probers), and safety review. In an evaluation using 120 samples from four families including Mirai and Gafgyt, matching responses were confirmed for approximately 70% of the Probers that established a TCP connection, showing that LLM-based extraction of communication behavior is effective for identifying malicious servers based on actual communication. Furthermore, probing with the generated Probers confirmed that candidate malicious servers can be discovered that are not limited to the known destinations contained in the input samples. On the other hand, it was also found that handling communication involving encryption or the derivation of decryption keys remains a challenge.

1. はじめに

IoT 機器を標的とするマルウェアはボットネットを形成し，DDoS 攻撃，認証情報の窃取，感染拡大などに悪用されている．このような被害を抑制するためには，感染端末の検出だけでなく，C&C サーバやローダなどの悪性サーバを早期に把握し，対策につなげることが重要である．

悪性サーバを検出するためには，マルウェア（ボット）と悪性サーバの間で行われる通信を把握し，その通信を再現・検証する必要がある．従来研究では，主に動的解析を利用した悪性サーバ検出手法が提案されてきた．しかし，動的解析では，実行環境の違い，悪性サーバの停止，解析

[1] NTT 株式会社 社会情報研究所
NTT Social Informatics Laboratories
[a)] kaz.shinomiya@ntt.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

回避機構などにより，通信挙動を十分に観測できるとは限らない．また，再現した通信が悪性サーバの識別の有効であるかの判定や，その判定根拠の説明性付与も課題である．

近年，大規模言語モデル（LLM）および関連技術の発展により，バイナリ解析ツールを操作する LLM エージェントを用いて，マルウェアの静的解析を自動化することが可能になりつつある．これにより，IoT マルウェアから，悪性サーバとの通信に関するフィンガープリント，すなわち通信先，通信プロトコル，メッセージ形式，応答判定条件などを自動的に抽出できる可能性が示されている [1]．

本研究では，LLM エージェントによる IoT マルウェアの静的解析結果に基づき，マルウェアと悪性サーバ間の通信をフィンガープリントとして抽出し，悪性サーバ探索に利用する手法を提案する．LLM エージェントによる解析結果には，誤解析やハルシネーションが含まれる可能性があるため，抽出されたフィンガープリントとその根拠を別の LLM エージェントが照合し，解析結果の妥当性を判定する．さらに，抽出したフィンガープリントの正確性を動的に評価するため，フィンガープリントを再現する探査プログラム（Prober）を生成し，実通信に基づいてフィンガープリントが有効であるかを検証する．Prober の生成では，通信処理を実装する生成エージェントと，生成された Prober が攻撃実行，認証試行，感染誘導などにつながる侵襲的な通信を発生させないかを審査するエージェントを用いる．これにより，攻撃行為の実行や侵襲性のあるパケット送信を防げるか検証する．また，ターゲット IP アドレス空間に対して Prober を実行し，応答がフィンガープリントと一致するホストを探索することで，既知の通信先に限定されない悪性サーバの発見が可能であることを示す．

本研究の主な貢献は以下の通りである．

(1) LLM マルチエージェントを用いて，IoT マルウェアの通信解析から Prober 生成，悪性サーバ探索までを自動化するシステムを提案・実装し，悪性サーバ探索に利用可能であることを示した．
(2) LLM エージェントによる IoT マルウェアの静的解析結果の正確性を，実通信の動的検証により評価した．
(3) LLM エージェントによる Prober 生成の安全性と有効性を評価した．

2. 背景と課題

2.1 関連研究

2.1.1 能動的探索による悪性サーバ探索

CyberProbe [2] は，既知の悪性サーバとの通信ログから対象ファミリ固有のフィンガープリントを生成し，探索対象ホストを悪性または良性に分類する手法である．13 種類の悪性ファミリに対して 23 個のフィンガープリントを生成し，C&C サーバやエクスプロイトサーバなどを発見できることが示されている．一方で，観測済みの通信トレースを前提とするため，未観測の通信や，C&C サーバとの通信を観測・再利用できない場合には，フィンガープリント生成が困難となる．また，時刻，実行環境，端末情報などに依存する通信では，過去に観測されたリクエストの単純な再送が成立しない可能性がある．

AutoProbe [3] は，テイント解析や記号的実行などの動的バイナリ解析により，悪性サーバ探索用のフィンガープリントを自動生成する手法である．リクエスト生成処理やレスポンス検証処理を解析し，プローブ生成関数，ポート選択関数，分類関数を構築することで，既知のライブ C&C サーバが存在しない場合でも検体からフィンガープリントを生成できる．一方で，動的解析に依存するため，マルウェアを適切に実行できる環境が必要である．特に IoT マルウェアでは，CPU アーキテクチャ，実行環境，解析回避処理などの影響により通信処理まで到達できない場合がある．また，観測範囲が実行経路に依存するため，通信仕様を網羅的に抽出できるとは限らない．

C2Miner [4] は，IoT マルウェアをサンドボックス内で実行し，生成された C&C 通信を候補 IP アドレス空間へリダイレクトすることで，C&C サーバを発見する手法である．その後，通信パターンに基づくフィンガープリントを用いて，C&C 通信の識別やクラスタリングを行う．一方で，この手法もマルウェアの動的実行を前提とするため，検体が正しく起動し，C&C 通信を生成する必要がある．また，フィンガープリントは主にパケット長と制御フラグに基づくため，コマンドの意味や応答構造といったアプリケーション層の内容は扱われない．さらに，探索には実マルウェアの通信を利用するため，安全性や倫理的制約を管理する必要がある．

2.1.2 LLM による自動解析

先行研究 [1] では，LLM エージェントが MCP 経由でバイナリ解析ツールを操作し，IoT マルウェアの静的解析から通信挙動を抽出する手法を提案している．この手法は，サンドボックス解析や人手による静的解析と比較しても遜色のない解析結果を出力できることが示されており，従来は専門家が手作業で行っていたバイナリ解析を自動化し，大量の検体から通信挙動を抽出できる可能性がある．そのため，抽出された通信挙動は悪性サーバ探索に必要なフィンガープリントの自動生成に利用できる可能性がある．

また，CHASE [5] は，悪性 PyPI パッケージを対象として，複数の LLM エージェントとツールを組み合わせることで，悪性コードの解析と説明生成を行うシステムである．マルチエージェント構成により解析タスクを分解し，復号や外部情報取得などの処理を専用ツールに委譲することで，LLM のハルシネーションや高エントロピー文字列処理の不安定性を低減している．

© 2026 Information Processing Society of Japan

2.2 課題

悪性サーバ探索に関する既存研究 [2], [3], [4] は，能動的探索の有効性を示しているが，観測済み通信やマルウェアの動的実行に依存している．IoT マルウェアでは，CPU アーキテクチャ等の実行環境，解析回避処理，外部サーバの停止などにより，動的解析で通信を観測できない場合がある．したがって，静的解析を起点として悪性サーバ探索に利用可能な通信特徴を抽出し，その妥当性を実通信に基づいて検証する枠組みが必要である．また，抽出した通信特徴を説明可能にすることで，人手による確認や安全な Prober 生成につなげることも重要である．

一方で，LLM による静的解析結果には，誤解析やハルシネーションが含まれる可能性がある．そのため，抽出された通信フローが実際の悪性サーバとの通信として成立するかを，動的に検証する必要がある．また，LLM が抽出した通信特徴を，既知の通信先の説明にとどめず，悪性サーバ探索に応用できるかは十分に明らかになっていない．

2.3 本研究の位置付けと目的

本研究では，LLM エージェントを用いた静的解析により IoT マルウェアからフィンガープリントを抽出し，その通信を再現する探査プログラム（Prober）を生成することで，悪性サーバ探索に利用する手法を提案する．本研究は，観測済み通信やマルウェアの動的実行に依存せず，LLM エージェントによる静的解析を起点としてフィンガープリントを抽出し，その妥当性を実通信に基づく動的検証により評価する点に特徴がある．また，Prober 生成において安全性評価エージェントを導入することで，攻撃行為や認証試行などを伴う侵襲的な通信を防ぎつつ，悪性サーバ探索に利用可能な通信を再現する．

3. 提案システム

図 1 に提案システムの概要を示す．提案システムは，マルウェア解析，Prober 生成，探索の 3 つから構成される．

本研究では，LLM エージェントが静的解析により抽出する構造化データを通信プロファイルと呼ぶ．通信プロファイルは，通信フロー，ステップ，通信先，メッセージ形式，応答判定条件，および根拠情報から構成される．ここで，検体内に実装された一連の通信処理を同一セッションの文脈で束ねた論理単位を通信フロー，通信フローを構成する個々の送受信操作をステップと呼ぶ．また，通信プロファイルに含まれる情報のうち，Prober が再現・照合し，悪性サーバ識別に用いる通信特徴をフィンガープリントと呼ぶ．

マルウェア解析では，LLM マルチエージェントにより検体を静的解析し，悪性サーバとの通信に関する通信プロファイルを抽出する．Prober 生成では，抽出された通信プロファイルに基づき，通信を再現する Prober を生成する．探索では，IP アドレスとポート番号からなるターゲット空間に対して Prober を実行し，応答が通信プロファイルと一致するホストを悪性サーバ候補として出力する．なお，検出可能な悪性サーバの数は，入力として与える IP アドレスとポート番号の集合に依存するため，本研究ではターゲット空間の生成や最適化は評価対象外とし，生成された Prober が悪性サーバ探索に利用可能であるかを検証することに焦点を当てる．各 LLM エージェントの実装には Claude Code を用い，モデルには Claude Opus 4.8 を 1 M トークンのコンテキスト長で使用した．

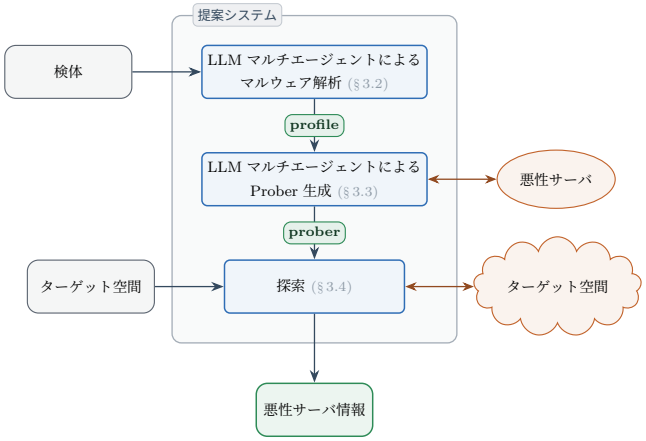


図 1: 提案システムの概要

3.1 システム概要

3.2 マルウェア解析

本研究では，解析を行う LLM エージェントを Analyzer，根拠照合を行う LLM エージェントを Analysis Reviewer と呼ぶ．複数の Analyzer が独立に通信プロファイルを抽出し，Analysis Reviewer がコード上の根拠と照合することで，根拠のない出力や不整合を低減する．

先行研究で示されているように，LLM エージェントによるマルウェア解析は，人手による解析に近い性能を示す一方で，誤解析やハルシネーションを含む可能性がある．そこで本研究では，LLM 出力の信頼性を向上させるため，複数の LLM インスタンスに独立した推論を行わせる研究 [6], [7] を参考にし，複数の解析結果を統合することで解析漏れを抑制する．また，生成結果を別過程で検証・修正する研究 [8], [9] を参考にし，LLM エージェントによる根拠照合を行う．

3.2.1 Analyzer

Analyzer は，MCP を介してバイナリ解析ツールを操作し，検体に含まれる外部サーバとの通信処理を解析し，通信プロファイルとして出力する．Analyzer が出力する通信プロファイルの例を付録 A に示す．

実装は，先行研究 [1] を基に行う．解析対象の IoT マルウェア検体を入力とし，プロンプトとともに LLM エージェ

© 2026 Information Processing Society of Japan

ントを起動すると，Ubuntu ベースの Docker コンテナが生成される．生成されたコンテナには検体がマウントされ，バイナリ解析ツール群が MCP サーバとして起動する．Analyzer は，これらのツール群を MCP を介して操作しながら検体を解析し，通信プロファイルを出力する．MCP ツールには，Ghidra [10] および Arkana [11] を使用した．使用したプロンプト例の抜粋を付録 B.1 に示す．

3.2.2 Analysis Reviewer

Analysis Reviewer は，Analyzer が出力した各通信プロファイルと検体を入力とし，通信プロファイルに含まれる通信仕様とコード上の根拠の整合性を確認する．根拠が十分であり，かつ整合している候補は採用し，根拠不足や不一致がある候補は棄却して後段の Prober 生成対象から除外する．これにより，単一 LLM エージェントによる解析で生じ得るハルシネーションや不整合を低減する．使用したプロンプト例の抜粋を付録 B.2 に示す．

3.3 Prober 生成

3.3.1 Prober 生成の目的と概要

本研究では，通信相手が悪性サーバであるかを判定する探査プログラムを Prober と呼ぶ．Prober 生成では，マルウェア解析により得られた通信プロファイルを入力として，悪性サーバ識別に用いる確認通信プログラムを生成する．

Prober を生成する目的は，静的解析により抽出された通信仕様が，実際に稼働中の C&C サーバやローダサーバの識別に利用可能であるかを検証することである．また，C&C サーバから攻撃命令を受信した場合や，ローダサーバから追加ペイロードの取得先が返された場合でも，攻撃実行や感染拡大に加担しない形で応答を観測することを目的とする．さらに，生成した Prober を用いることで，既知の通信先に限定されない悪性サーバ探索へ接続できる．

Prober 生成では，通信プロファイルに基づいて，探査に必要な通信挙動をプログラムとして実装する．これにより，抽出された通信プロトコルや応答判定条件を明確化するとともに，実マルウェアを動作させずに悪性サーバからの応答を確認できる．また，生成された Prober は，通信プロファイルに含まれるメッセージ形式，応答判定条件，および根拠情報に基づいて動作する．そのため，Probing により応答一致が確認された場合には，どの通信特徴に一致したか，またその特徴が検体中のどのコードや定数に由来するかを対応付けて説明できる．

一方で，Prober 生成では，攻撃行為，認証試行，感染誘導などを伴う侵襲的な通信の発生を抑制する必要がある．そこで本研究では，Generator が Prober を生成し，Safety Reviewer が生成された Prober の送信内容や処理を確認する構成とする．これにより，安全性に問題のある Prober の生成および実行を抑制する．

3.3.2 Generator

Generator は，通信プロファイルに含まれるプロトコル，ポート番号，メッセージ形式，応答判定条件に基づき，悪性サーバ識別に必要な通信処理のみを実装する．通信プロファイルには，悪性サーバとの通信以外に，スキャンや攻撃通信などが含まれる場合がある．そのため，通信プロファイル中に記載された通信種別に基づき，Prober として実装すべき通信フローを識別する．使用したプロンプト例の抜粋を付録 B.3 に示す．

3.3.3 Safety Reviewer

Safety Reviewer は，Generator により生成された Prober の安全性を確認する．具体的には，Menlo Report [12] を参考に，研究倫理節で述べる条件を満たし，悪性サーバ識別に必要な最小限の通信に限定されているかを確認する．使用したプロンプト例の抜粋を付録 B.4 に示す．

3.3.4 Prober の実行

安全性審査を通過した Prober については，既知の悪性サーバまたは検証環境に対して実行し，応答判定条件を確認する．この検証により，通信プロファイルから生成された Prober が実際のサーバ応答を識別可能であるかを確認する．最終的に，有効性と安全性が確認された Prober のみを後段の悪性サーバ探索に用いる．

3.4 探索

本研究では，探索対象となる IP アドレスおよびポート番号の集合をターゲット空間と呼ぶ．事前に定義したターゲット空間に対して Prober を実行し，得られた応答を通信プロファイルに含まれる応答判定条件と照合することで，既知の通信先に限定されない悪性サーバ候補を探索する．

ターゲット空間の生成および最適化による探索効率の改善は，本研究のスコープ外である．そのため本研究では，C2Miner [4] を参考に，VirusTotal で確認された過去の悪性サーバ情報を用いてターゲット空間を生成する．具体的には，各ファミリについて，過去の悪性サーバの IP アドレスが多く観測された上位 3 つの AS を選択し，各 AS 内で悪性サーバ数が最も多い/24 サブネットを 1 つずつ選択する．最後に，選択した/24 サブネット内の IP アドレスと，マルウェア解析で得られたポート番号の直積を取り，探索対象となる IP:port 集合を生成する．

なお，研究倫理節で述べる通り，第三者への影響を抑えるため，探索には Safety Reviewer による安全性審査を通過し，さらに人手で確認した Prober のみを使用する．また，通信回数，送信サイズ，実行頻度を制限し，攻撃命令の実行，認証試行，感染誘導，ペイロード取得，サーバ状態の変更を伴う通信は行わない．

© 2026 Information Processing Society of Japan

4. 評価方法

4.1 評価設定

本研究では，提案手法の性能と特性を以下の 3 つの研究課題（Research Question）に基づいて評価する．

RQ1: マルウェア解析による悪性通信抽出の性能 LLM エージェントを用いたマルウェア解析において，通信プロファイルをどの程度正確に抽出できるか．また，マルチエージェント構成および Analysis Reviewer の導入により，抽出性能がどのように変化するか．

RQ2: Prober 生成の性能 LLM エージェントを用いた Prober 生成において，通信プロファイルに基づく Prober を正しく生成できるか．また，Safety Reviewer により，侵襲的な Prober の生成を抑制できるか．

RQ3: 悪性インフラ探索 生成された Prober を用いることで，既知の通信先に含まれない悪性サーバ候補を発見できるか．

4.2 データセット

評価データセットの内訳とアンパック結果を表 1 に示す．評価検体は，MalwareBazaar [13] から取得し，Signature 検索機能を用いて各ファミリの検体を収集した．

IoT マルウェアには，解析妨害を目的として UPX で圧縮された検体が存在する．本研究ではアンパック済み検体を入力とするため，Analyzer と同様の環境を用いて標準 UPX および改変 UPX 検体をアンパックした．その後，非パッキング検体およびアンパック済み検体から無作為に計 4 ファミリ 120 検体を選定し，評価に用いた．

表 1: 収集検体のアンパック結果と評価データセット内訳

	収集検体のアンパック結果				評価データセット（検体数）				
ファミリ	収集数	UPX	改変 UPX	成功	x86_32	x86_64	ARM	MIPS	計
Mirai	173	10	7	17	14	12	3	3	30
Gafgyt	158	9	7	16	9	7	12	3	30
XorDDoS	81	0	0	0	26	4	0	0	30
Tsunami	133	0	1	1	7	3	10	10	30
計	545	19	15	34	56	26	25	16	120

5. 評価結果

表 2 に Reviewer およびマルチエージェント構成の有無による提案システム各段階の抽出数の比較を示す．

抽出通信フロー数は，Analyzer によるマルウェア解析の結果として抽出された，悪性サーバとの通信フローの数を表す．レビュー通過数は，Analysis Reviewer による根拠照合の結果，コード上の根拠が確認され，後段の Prober 生成に利用可能であると判定された通信フローの数である．Prober 生成数は，Generator により，レビューを通過した通信フローに基づき，通信を再現する Prober を生成できた数を示す．安全性レビュー通過数は，生成された Prober のうち，Safety Reviewer により安全性が確認された数である．人手による確認通過数は Safety Reviewer による確認を通過した Prober を人手で安全性が確認された数を表す．Probing 成立数は，安全性レビューを通過した Prober を実行した結果，対象サーバから応答を得られ，かつ応答が抽出された通信特徴の判定条件と一致した数を表す．

また表 3 に，Prober の実行結果の内訳を示す．複数の Analyzer による解析に加えて，Reviewer による確認を行った場合に，Prober を解析結果の通信先に対して実行した際の挙動を示している．なお，Analyzer の数は 3 とした．

5.1 RQ1: マルウェア解析による悪性通信抽出の性能

5.1.1 マルウェア解析による悪性通信抽出の性能

表 3 より，TCP 接続が成立した Prober のうち，応答一致まで確認できたものは全体で約 70%（16/23）であった．Mirai，Gafgyt，Tsunami のように，独自のバイナリプロトコルや IRC を用いた平文のプロトコルでは，高い応答一致率が得られた．

なお，本割合は TCP 接続が成立した Prober 数を母数とし，悪性サーバの停止，名前解決不能，および通信先取得失敗などにより TCP 接続が成立しなかったケースは，通信仕様抽出の正確性とは区別して扱う．

応答が一致しなかった検体では，送信メッセージは平文であるものの，サーバからの応答に暗号化が施されており，Prober に実装された応答判定条件と一致しない事例が確認された．具体的には，XOR 符号化や ChaCha 等の暗号化を用いる検体が含まれていた．復号に用いる鍵がファミリ内で共通している場合には応答一致に成功するケースがあったが，検体ごとに異なる暗号化処理や鍵生成処理を含む場合には，成功率が低下する傾向が見られた．また，応答が一致したケースにおいても，任意の文字列を受理する正規表現が実装されている事例が確認された．このことから，Prober により応答の有無は確認できる一方で，暗号化通信を含む検体では応答内容の厳密な解釈が課題となる．

5.1.2 LLM エージェント構成の比較

マルチエージェント構成により解析漏れを抑制し，Reviewer エージェントにより根拠照合によりハルシネーションや誤解析を低減することを検討した．

単一の LLM エージェントによる解析結果をそのまま用いた場合 (R ✗, M ✗) では，Reviewer による根拠照合を行わないため，根拠が不十分な通信仕様や，Prober 生成に必要な情報が欠落した通信フローが後段に残る場合があった．

複数の LLM エージェントが独立に解析を行った場合 (R ✗, M ✓) では，複数の解析結果を用いることで，単一エージェントでは見落とされた通信を抽出できる場合があった．一方で，Reviewer による根拠照合を行わないため，エー

© 2026 Information Processing Society of Japan

表 2: Reviewer およびマルチエージェント構成の有無による提案システム各段階の抽出数の推移の比較

ファミリ	検体数	(R ✗, M ✗)						(R ✗, M ✓)						(R ✓, M ✗)						(R ✓, M ✓)						
		①	②	③	④	⑤	⑥	①	②	③	④	⑤	⑥	①	②	③	④	⑤	⑥	①	②	③	④	⑤	⑥	⑦
Mirai	30	40	–	33	33	33	5	44	–	44	44	44	7	40	35	35	35	35	5	44	37	37	37	37	7	1
Gafgyt	30	36	–	30	30	30	3	38	–	38	38	38	5	36	30	30	30	30	3	38	35	35	35	35	5	1
Tsunami	30	34	–	30	30	30	1	38	–	38	38	38	3	35	30	30	30	30	1	38	32	32	32	32	3	0
XorDDoS	30	33	–	30	30	30	0	39	–	39	39	39	1	32	30	30	30	30	0	39	35	35	35	35	1	0
合計	120	143	–	123	123	123	9	159	–	159	159	159	16	143	125	125	125	125	9	159	139	139	139	139	16	2

R：Analysis Reviewer の有無，M：マルチエージェント構成の有無，①：抽出通信フロー数，②：レビュー通過数，
③：Prober 生成数，④：安全性レビュー通過数，⑤：人手による確認通過数，⑥：Probing 成立数，⑦：探索による発見数

表 3: Prober の実行結果の内訳

ファミリ	Prober 生成数	HS 成立	応答あり	応答一致
Mirai	37	9	9	7
Gafgyt	35	7	6	5
Tsunami	32	3	3	3
XorDDoS	35	4	3	1
合計	139	23	21	16

表 4: Analyzer エージェント数によるレビュー通過数比較

エージェント数	1	2	3	5	10
Mirai	35	36	37	38	38
Gafgyt	30	31	35	36	36
Tsunami	30	30	32	33	33
XorDDoS	30	30	30	30	30
合計	125	127	134	137	137

ジェント間での矛盾や根拠が不十分な通信仕様も残った．

単一エージェントによる解析結果に対して，Reviewer による根拠照合を行った場合 (R ✓, M ✗) では，Reviewer を導入することで，コード上の根拠が不十分な通信フローや，Prober 生成に必要な情報が欠落した通信フローを除外できた．また，Reviewer なしの場合と比較して Probing 成立数およびその内容が同一であったことから，Reviewer による有効な通信フローの誤棄却は確認されなかった．

複数の LLM エージェントによる独立解析に加えて，Reviewer による根拠照合を行った場合 (R ✓, M ✓) では，複数エージェントにより通信フローの抽出範囲を広げつつ，Reviewer により根拠のない通信仕様や不整合を除外できる．したがって，抽出通信フロー数と信頼性の両面から，最も安定した構成であると考えられる．

以上より，マルチエージェント構成では，単一エージェント構成と比較して抽出通信フロー数および Probing 成立数が増加した．また，Reviewer の導入は根拠不足の通信フローを除外することで，後段の Probing の失敗率を低減し，無駄な Prober 実行を抑制する効果が見られた．これにより，不要な Prober 生成および実ネットワークへの不要な接続試行を抑制でき，探索時間や計算資源の削減に加え，第三者への不要な通信，誤検知，および安全性審査の負荷を低減できる．

5.1.3 エージェント数による比較

表 4 に，Analyzer エージェント数を変化させた場合のレビュー通過数の比較結果を示す．エージェント数を増やすことで，単一エージェントでは見落とされた通信フローを抽出でき，取り逃がしを抑制できる傾向が見られたが増加幅は小さく，抽出数には一定の頭打ちが見られた．また，エージェント数の増加に伴い，解析時間やトークン数などのコストも増加するため，対象検体数や解析目的に応じて適切なエージェント数を設定する必要がある．本研究では，抽出性能とコストのバランスを考慮し，以降の実験では Analyzer エージェント数を 3 に設定した．

表 5: 1 検体あたりのコスト

指標	Analyzer	Analysis Reviewer	Generator	Safety Reviewer
時間 [s]	1,188	298	143	62
コスト [$]	8.66	1.56	0.49	0.18
ターン数	90	36	–	–

5.1.4 解析コスト

提案手法による 1 検体あたりの解析コストを表 5 に示す．ここでターン数とは，1 検体の解析中に LLM エージェントが推論とツール呼び出しを繰り返した回数である．特に Analyzer において大きな値を示し，LLM の推論と MCP ツール群を用いた反復的な静的解析を要した．

5.2 RQ2: Prober 生成の性能

5.2.1 Prober 生成

Prober 生成では，マルウェア解析によって得られた通信プロファイルを入力として，悪性サーバの識別に必要な通信を再現する探査プログラムを生成する．表 2 に示すように，評価対象としたすべての入力に対して Prober 生成に成功し，人手による確認においても入力されたフィンガープリントと異なる通信を行う Prober は確認されなかった．

© 2026 Information Processing Society of Japan

5.2.2 Prober の安全性検査

Prober の安全性検査では，生成された Prober を Safety Reviewer により検査し，攻撃行為，認証試行，感染誘導，ペイロード取得などを伴う侵襲的な処理が含まれていないかを確認した．表 2 に示すように，本評価で生成された Prober に対して，Safety Reviewer により安全性上の問題が検出された事例はなかった．また，人手による確認においても，侵襲的な処理を含む Prober は確認されなかった．

さらに追加実験として，人手により作成した 5 件の侵襲的な通信フローを入力し，Safety Reviewer がそれらを検知できるかを評価した．その結果，Safety Reviewer はすべての入力について侵襲的な処理を検知できた．この結果から，Safety Reviewer は，少なくとも本研究で定義した危険な通信パターンに対して，Prober の実行前検査として有効であることが確認された．

5.2.3 Prober の実行

表 3 に示したとおり，応答一致まで確認できた Prober は 16 件であった．ハンドシェイクが失敗したケースは，マルウェアの取得から Probing の実行までの間に悪性サーバが停止または移転していたことに加え，Dynamic DNS を用いるドメインが名前解決不能となっていたこと，および EtherHiding によりスマートコントラクト上に格納された IP アドレスの取得の失敗に起因するものであった．

悪性サーバ候補の多くは C&C サーバであったが，マルウェア検体や追加ペイロードを配布するローダサーバも含まれていた．また，同一ホスト上でポート番号を変え，C&C 通信，検体配布，追加ペイロード配布など複数の役割を担うケースも確認され，これらの重複を除くと，悪性サーバの IP アドレス数は 11 であった．

5.3 RQ3: 悪性インフラ探索

5.3.1 探索結果

探索では，各ターゲットに対して Prober を実行し，応答が通信プロファイルに含まれる応答判定条件と一致するかを確認した．応答判定条件を満たしたターゲットを悪性インフラ候補として記録し，入力検体に含まれる既知の通信先に含まれない候補を新規候補として扱った．

表 2 に示した通り，Mirai および Gafgyt 検体を入力として生成した Prober により，検体にハードコードされていない悪性サーバ候補を発見できることを確認した．この結果は，提案システムにより生成された Prober が，既知の通信先の再確認にとどまらず，悪性インフラ探索にも利用可能であることを示している．本研究ではターゲット空間の生成や最適化は評価対象外としているが，ターゲット空間の拡大や選定手法の高度化により，より多くの悪性サーバ候補を発見できる可能性がある．

5.3.2 脅威インテリジェンスとの照合結果

探索によって発見された新規の悪性インフラ候補について，VirusTotal に登録された情報と照合した．その結果，発見された候補には，VirusTotal 上で検知エンジン数が 3 以下の IP アドレスが含まれていた．このことから，提案手法は既存の外部脅威インテリジェンスで十分に検知されていない悪性サーバ候補の早期発見に役立つ可能性がある．

6. 考察と課題

6.1 考察

6.1.1 RQ1: マルウェア解析による悪性通信抽出の性能

本研究のデータセットを用いた評価では，TCP 接続が成立した Prober のうち約 70%で応答一致を確認した．また，マルチエージェント構成により解析漏れを抑制し，Reviewer エージェントにより根拠照合によりハルシネーションや誤解析を低減することが確認できた．応答一致に失敗した事例は，主に暗号化や復号処理を含む通信に対して十分に対応できていないことに起因すると考えられる．そのため，暗号化処理や復号鍵導出処理の抽出，および応答内容の解釈精度の改善が今後の課題である．

6.1.2 RQ2: Prober 生成の性能

LLM エージェントにより，通信プロファイルから実行可能な Prober プログラムを生成できた．実際に生成された Prober に，攻撃行為，認証試行，感染誘導，ペイロード取得などを伴う侵襲的な処理は確認されなかった．また，侵襲的な処理を含む Prober を意図的に入力する追加実験により，Safety Reviewer が安全性に問題のある Prober を検出し，後段の実行を抑制できることを確認した．この結果から，LLM エージェントによる Prober 生成において，Safety Reviewer を導入することは安全性確保に有効であると考えられる．また，応答一致が確認された候補については，対応する付録 A のような通信プロファイルを参照することで，判定に用いられた通信特徴が確認できる．

さらに，生成された Prober は，後述の探索だけでなく，マルウェア感染疑いのあるホストの検査にも応用可能であると考えられる．

6.1.3 RQ3: 悪性インフラ探索

生成された Prober を用いることで，既知の通信先に含まれない悪性サーバ候補を発見できることを確認した．この結果は，提案システムにより，マルウェア検体の静的な通信解析から Prober 生成，悪性インフラ探索までを一連の流れとして接続できることを示している．特に，動的解析で通信を十分に観測できない場合でも，静的解析により得られた通信プロファイルを起点として，悪性インフラ探索に利用可能な確認通信を生成できる．

© 2026 Information Processing Society of Japan

6.2 今後の課題

6.2.1 暗号化対応と応答を入力とした再解析

暗号化処理や復号鍵導出処理の抽出は，課題の一つであるが，観測された応答を LLM エージェントに入力して再解析することで，応答の意味や復号方法を解釈できるケースも確認された．高エントロピー文字列の転記，変換，復号，照合を LLM 単体に依存すると誤りが生じやすいことが知られているため，Prober 実行結果を解析エージェントへフィードバックしつつ，復号や照合などの処理は決定的な補助ツールで検証し，通信プロファイルや応答判定条件を更新する反復的な解析手法を検討する必要がある．

6.2.2 コスト

本研究の手法は，1 検体あたりの解析時間および API コストが高い．特に，複数の Analyzer エージェントによる検体解析や Reviewer エージェントによる根拠照合は，信頼性向上に有効である一方で，トークン数と実行時間の増加につながる．そのため，オープンウェイトモデルを用いたローカル環境での LLM エージェント実行や，小型モデルと高性能モデルの使い分けにより，コストの低減を検討する必要があるただし，モデルの変更は解析精度や安全性審査の性能に影響するため，性能面での評価が必要である．

6.2.3 LLM の利用ポリシーによる解析停止

本研究で用いたモデルには，サービス提供者の利用ポリシーに基づくフィルタが設けられており，解析が途中で停止する事例が観察された．この制約は LLM をセキュリティ研究に応用する際の課題であり，モデル選定，プロンプト設計，出力形式の制御等の緩和策の検討が必要である．

6.2.4 ターゲット空間生成による探索効率化

本研究では，ターゲット空間の生成や最適化は主たる評価対象としていない．しかし，過去の悪性インフラに関する情報に加えて，外部脅威インテリジェンスを組み合わせることで，より多くの新規悪性インフラを早期に発見できる可能性がある．また，提案システムを長期的に運用し，発見結果を次回以降のターゲット空間生成にフィードバックすることで，探索効率を改善できる可能性がある．

6.2.5 発見された悪性サーバ群の分析

提案システムにより発見された悪性サーバ候補を継続的に分析することで，悪性サーバの移転，ファミリ間の関係，通信プロトコルの共通性，ホスティング事業者や ASN の偏りなどを明らかにできる可能性がある．これらの分析結果を利用し，防御運用や早期対策に活用する方法を検討する．

7. おわりに

本研究では，LLM エージェントが MCP を介してバイナリ解析ツールを操作し，IoT マルウェア検体から抽出した通信プロファイルに基づいて生成された探査プログラムによって悪性サーバ探索に利用可能であることを示した．また，Analyzer の複数化により解析漏れを抑制し，Analysis Reviewer による根拠照合により誤解析やハルシネーションを低減できることを示した．加えて，Safety Reviewer により，侵襲的な通信を抑制した Prober 生成が可能であることを確認した．

評価の結果，TCP 接続が成立した Prober のうち約 70 %で応答一致を確認し，LLM の静的解析による通信仕様抽出が，実通信に基づく悪性サーバ識別に有効であることを示した．また，通信プロファイルに含まれる根拠情報を参照することで，Probing による判定理由を説明できる．

今後の課題は暗号化や復号鍵導出処理を含む通信への対応，ターゲット空間生成の高度化，および発見された悪性サーバ候補と外部脅威インテリジェンスの連携である．これらにより，IoT マルウェアを起点としたサイバー攻撃の早期把握と抑制に貢献することを目指す．

参考文献

[1] 篠宮一真，岩村誠，“LLM エージェントを用いたバイナリ解析による IoT マルウェアの通信挙動抽出,” 電子情報通信学会 第 114 回 CSEC 研究発表会, 2026.

[2] Nappa, A. et al.: CyberProbe: Towards Internet-Scale Active Detection of Malicious Servers, Proc. 21st Network and Distributed System Security Symposium, 2014.

[3] Xu, Z. et al.: AUTOPROBE: Towards Automatic Active Malicious Server Probing Using Dynamic Binary Analysis, Proc. ACM SIGSAC Conference on Computer and Communications Security, 2014.

[4] Davanian, A. et al.: C2Miner: Tricking IoT Malware into Revealing Live Command & Control Servers, Proc. ACM Asia Conference on Computer and Communications Security, 2024.

[5] Toda, T. and Mori, T.: CHASE: LLM Agents for Dissecting Malicious PyPI Packages, Proc. IEEE/ACM International Conference on AI-powered Software, 2025.

[6] Du, Y. et al.: Improving Factuality and Reasoning in Language Models through Multiagent Debate, Proc. International Conference on Machine Learning, 2024.

[7] Liang, T. et al.: Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate, Proc. Conference on Empirical Methods in Natural Language Processing, 2024.

[8] Dhuliawala, S. et al.: Chain-of-Verification Reduces Hallucination in Large Language Models, Findings of the Association for Computational Linguistics: ACL, 2024.

[9] Gou, Z. et al.: CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing, International Conference on Learning Representations, 2024.

[10] National Security Agency: Ghidra: A Software Reverse Engineering Framework, `https://github.com/NationalSecurityAgency/ghidra`.

[11] Brine, J.: Arkana: 250+ MCP Tools for Malware Analysis, `https://github.com/JameZUK/Arkana`.

[12] Dittrich, D. and Kenneally, E.: The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research, U.S. Department of Homeland Security, Science and Technology Directorate, 2012.

[13] abuse.ch and Spamhaus: MalwareBazaar, `https://bazaar.abuse.ch/`.

[14] VirusTotal, `https://www.virustotal.com/`.

© 2026 Information Processing Society of Japan

研究倫理

本研究では，Prober を用いて実ネットワーク上のサーバに通信を行うため，Menlo Report [12] を参考に，第三者への影響を最小化するよう実験を設計した．探査対象となるサーバの管理者およびネットワーク運用者を，影響を受け得るステークホルダとして考慮し，Prober 実行前には LLM エージェントによる安全性審査に加えて人手による確認を行った．実験では，悪性サーバ識別に必要な最小限の通信のみを発生させるため，以下の制約を設けた．

- 応答に攻撃命令が含まれる場合でも，記録にとどめ，実行しない．
- スキャン，DDoS，感染拡大，コマンド実行などの攻撃機能を実装しない．
- マルウェア本体や追加ペイロードの取得，配置，実行を促す感染誘導を行わない．
- ペイロード本体や追加モジュールを保存しない．
- 通信回数，送信サイズ，実行頻度を最小限に制限する．

サーバ応答に新たな通信先，URL，コマンド，ペイロード情報が含まれる場合でも，それらに基づく追加通信や取得処理は行わず，記録と照合のみに用いる．

探索対象は，既知の悪性サーバ，関連するポート番号，外部脅威インテリジェンスなどの根拠に基づいて選定し，無差別な広域探索は行わない．対象が良性ホストであった場合であっても，送信するパケットは一般的なネットワークサービスとの疎通を目的としたものではなく，通信量も初期パケット送信および必要最小限の確認通信に限定されるため，サービスに影響を与える可能性は低いと考えられる．探索対象への個別の事前通知は，攻撃者に観測手法や探索条件を知らせ，回避行動を誘発する可能性があるため行わない．また，良性ホスト管理者への実験後の個別通知についても，通知内容が攻撃者に伝わる可能性を考慮し，本研究では行わない．

サービス妨害とみなされる可能性を低減するため，複数 IP アドレスへのパケット送出は一定時間の間隔を空けて行い，判定完了後または一定時間経過後に接続を終了する．また，実験には仮想環境を用い，実験ごとに初期状態へ復元するとともに，探索対象として指定した IP アドレスおよびポート番号以外への外向き通信を制限した．予期しない大量応答，異常な通信量，または第三者への影響が疑われる挙動を確認した場合には，直ちに実験を停止し，対象を探索範囲から除外することとした．

付　　録

A　通信プロファイル例

A.1　Mirai 検体の通信プロファイルの例（抜粋）


```
{
  "flow_id": "cnc_session_domain_01",
  "flow_type": "c2_session",
  "flow_summary": "Main C2 session that connects to TCP/35342 at the DNS-resolved domain
       xxx.xxx.org, sequentially sends the magic value, architecture, version, CPU count,
        memory size, and bot ID for registration, and then receives length-prefixed
       commands.",
  "trigger": "After startup, when g_46a4a8 is disconnected (-1), the connection starts if
       rand()%100>=31 and DNS resolution succeeds.",
  "confidence": "high",
  "evidence": [
    {
      "kind": "bytes",
      "va": "0x4274e0",
      "expect_hex": "30aecc0e984f97c7669cffa07e283dea",
      "claim": "Leading ciphertext of the C2 domain (35 bytes). It decrypts to xxx.duckdns
        .org using ChaCha20 with counter=1, key 0x46a4b0, and nonce 0x46a4d0."
    },
    {
      "kind": "bytes",
      "va": "0x46a4b0",
      "expect_hex": "161e191b111f001d041c0e080b1a120705090d0f060a15010c141f1702031318",
      "claim": "The 32-byte ChaCha20 key used for string-table decryption and send-field
        encryption."
    },
    {
      "kind": "bytes",
      "va": "0x46a4d0",
      "expect_hex": "1e004a000000000000000000",
      "claim": "The 12-byte ChaCha20 nonce."
    },
    {
      "kind": "bytes",
      "va": "0x404624",
      "expect_hex": "3c026170",
      "claim": "Loads the constant 0x6170 (\"ap\", the upper part of \"expa\"), indicating
        that the decryption routine is ChaCha20."
    },
    {
      "kind": "bytes",
      "va": "0x400ca8",
      "expect_hex": "24028a0ea6e20002",
      "claim": "C2 port setup that stores 0x8a0e (35342) into sin_port."
    }
  ],
  "steps": [
    {
      "ip": "x.x.x.x",
      "port": 35342,
      "transport": "tcp",
      "l7_protocol": "custom_binary",
      "direction": "send",
      "payload": "^\\x00\\x00\\x00\\x04$",
      "category": "c2_control",
      "description": "The first fixed 4-byte magic value sent after a successful connect.
       The value at .rodata 0x427430 is sent as-is via send(fd, ptr, 4, MSG_NOSIGNAL) as
       the registration start marker.",
      "evidence": [
        {
          "kind": "bytes",
          "va": "0x427430",
          "expect_hex": "00000004",
          "claim": "The actual fixed 4-byte value at the beginning of the registration
       packet."
        },
        {
          "kind": "bytes",
          "va": "0x400f34",
          "expect_hex": "24a5743024060004",
          "claim": "The code location that sets a1=0x427430 and a2=4 before calling send."
        },
        {
          "kind": "bytes",
          "va": "0x400be8",
          "expect_hex": "24050002",
          "claim": "The second argument to socket() is 2, which corresponds to SOCK_STREAM
       in the MIPS ABI and indicates TCP."
        }
      ]
    },
    {
      "ip": "x.x.x.x",
      "port": 35342,
      "transport": "tcp",
      "l7_protocol": "custom_binary",
      "direction": "send",
      "payload": "^\\x04$",
      "category": "c2_control",
      "description": "Sends the length of the following architecture string as one byte.
       Since strlen(\"mips\")=4, this is always the fixed value 0x04.",
      "evidence": [
        {
          "kind": "bytes",
          "va": "0x400f54",
          "expect_hex": "27a50021",
          "claim": "Sets the stack location sp+0x21, where the length byte is stored, as
       the second argument to send."
        }
      ]
    },
    ...
  ]
}
```


© 2026 Information Processing Society of Japan

A.2 Gafgyt 検体の通信プロファイルの例（抜粋）


```
{
  "flow_id": "c2_session_register_and_command_loop",
  "flow_type": "c2_session",
  "flow_summary": "After main connects to x.x.x.x:5555 in commServer[0] via initConnection
       , it sends a registration line containing victim host information using sockprintf
       , and then maintains a persistent loop that receives newline-delimited commands
       via recvLine and passes them to processCmd.",
  "trigger": "Immediately after process startup, daemonization via fork/setsid, and entry
       into the main loop. Upon disconnection, it reconnects after sleep(5).",
  "confidence": "high",
  "evidence": [
    {
      "kind": "string",
      "va": "0x418630",
      "expect_str": "x.x.x.x:5555",
      "claim": "The C2 endpoint string referenced by commServer[0], which determines the
        destination IP and port."
    },
    {
      "kind": "bytes",
      "va": "0x45b990",
      "expect_hex": "30864100",
      "claim": "Shows that the first element of the commServer array in .data is a pointer
         to 0x418630, the C2 string."
    },
    {
      "kind": "symbol",
      "va": "0x40b994",
      "name": "initConnection",
      "claim": "Location of the C2 connection function that retrieves the destination from
         commServer and calls socket/connectTimeout."
    }
  ],
  "steps": [
    {
      "ip": "x.x.x.x",
      "port": 5555,
      "transport": "tcp",
      "l7_protocol": "custom_text",
      "direction": "send",
      "payload": "^\\x1b\\[1;95mUser: [!-~]{1,64} \\| IP:
        [0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3}\\.[0-9]{1,3} \\| Ports: [ -~]{1,512} \\| Arch
        : Mips \\| CPU: [ -~]{1,64} \\| Mem: [ -~]{1,64} \\| Sys: [ -~]{1,512} \\|\\x1b
        \\[1;32mUKdownload\\x1b\\[0m\\n$",
      "category": "report_exfil",
      "description": "Immediately after connection establishment, the results of getUser,
        inet_ntoa, getPortz, getArch, getCores, getMemory, and getSoftware are formatted
        into a single ANSI-colored line and sent as a registration report via sockprintf."
        ,
      "evidence": [
        {
          "kind": "string",
          "va": "0x419e4c",
          "expect_str": "\u001b[1;95mUser: %s | IP: %s | Ports: %s | Arch: %s | CPU: %s |
       Mem: %s | Sys: %s |\u001b[1;32mUKdownload\u001b[0m",
          "claim": "The actual registration-line format string passed to sockprintf,
       supporting the fixed parts of the payload."
        },
        {
          "kind": "insn",
          "va": "0x40bf10",
          "name": "jalr sockprintf",
          "claim": "The sockprintf call site in main that sends the registration line to
       mainCommSock."
        },
        {
          "kind": "string",
          "va": "0x4195d8",
          "expect_str": "Mips",
          "claim": "The fixed string returned by getArch, supporting that the Arch field
       in the payload becomes Mips."
        },
        {
          "kind": "symbol",
          "va": "0x401a3c",
          "name": "sockprintf",
          "claim": "Location of the sending routine that appends 0x0a to the formatted
       string and sends it using send(MSG_NOSIGNAL)."
        }
      ]
    },
    {
      ...,
      "direction": "recv",
      "payload": "^![!-~]{1,32} [A-Z]{3,7}( [!-~]{1,255}){0,8}\\r?\\n$",
      "category": "c2_control",
      "description": "recvLine receives data one byte at a time until a newline and treats
         it as one command. Only lines beginning with 0x21 are split into an ID and
       command word and passed to processCmd.",
      "evidence": [
        {
          "kind": "symbol",
          "va": "0x401f60",
          "name": "recvLine",
          "claim": "Location of the receive routine that repeatedly calls recv(fd,&c,1,0)
       after select and returns one line up to 0x0a."
        },
        {
          "kind": "insn",
          "va": "0x40c80c",
          "name": "jalr recvLine",
          "claim": "The only call site in the main loop that receives command lines from
       the C2."
        },
        ...
      ]
    }
  ]
}
```


A.3 XorDDoS 検体の通信プロファイルの例（抜粋）


```
{
  "flow_id": "c2_session_01",
  "flow_type": "c2_session",
  "flow_summary": "Persistent session that connects via TCP to the decrypted C2 xx.xxxxxxx
       .com:1526, sends a 28-byte header with CRC32 followed by 272 bytes of XOR-
       encrypted host information for registration, and then continuously receives
       command headers and payloads.",
  "trigger": "Started by the fcn.0804e140 thread created by main as it iterates over the
       C2 linked list. If a connection fails, it proceeds to the next entry.",
  "confidence": "high",
  "evidence": [
    {
      "kind": "bytes",
      "va": "0x80a67a8",
      "expect_hex": "28281c27204045777475175565b5c7c01",
      "claim": "The first 16 bytes of the encrypted C2 list. XORing them with the above
        key yields 'xxx.xxx.com:1'."
    },
    {
      "kind": "string",
      "va": "0x80c1888",
      "expect_str": "BB2FA36AAA9541F0",
      "claim": "The actual 16-byte XOR decryption key, referenced by fcn@0x08049130 as key
        [i%16]."
    },
    {
      "kind": "bytes",
      "va": "0x804e626",
      "expect_hex": "c7442404a81c0b08",
      "claim": "Instruction that pushes the C2 ciphertext address as the decryption source
         (referenced VA 0x080b1ca8 = file-backed 0x080a67a8)."
    },
    {
      "kind": "insn",
      "va": "0x804e5cc",
      "name": "mov word [edi+0x100], ax",
      "claim": "Instruction that stores the port parsed from 'host:port' into offset +0
       x100 of the C2 entry structure."
    }
  ],
  "steps": [
    {
      "ip": "x.x.x.x",
      "port": 1526,
      "transport": "tcp",
      "l7_protocol": "custom_binary",
      "direction": "send",
      "payload": "^\\x36\\xa0\\x26\\xb8\\x10\\x01\\x00\\x00[\\x00]{20}$",
      "category": "c2_control",
      "description": "Sends a 28-byte plaintext header immediately after TCP connection.
       The first 4 bytes are the CRC32 of the following 24 bytes (initial value 0
       xFFFFFFFF, no final XOR), and the next 4 bytes encode the body length 0x110.",
      "evidence": [
        {
          "kind": "insn",
          "va": "0x804e1d1",
          "name": "call tcp_connect",
          "claim": "Call site that connects to the IP/port of the C2 entry over TCP with a
       5-second timeout."
        },
        {
          "kind": "bytes",
          "va": "0x804e223",
          "expect_hex": "c745dc10010000",
          "claim": "Instruction that sets the second word of the header, the payload
       length, to the constant 0x110 = 272."
        },
        {
          "kind": "bytes",
          "va": "0x8048fc5",
          "expect_hex": "81f22083b8ed",
          "claim": "Polynomial 0xEDB88320 used for CRC32 table generation. The first 4
       bytes of the header are this CRC32 value (initial value 0xFFFFFFFF, no final XOR).
       "
        },
        {
          "kind": "insn",
          "va": "0x804e390",
          "name": "call send_all",
          "claim": "Call site that sends the 28-byte plaintext header."
        }
      ]
    },
    {
      ...,
      "direction": "send",
      "payload": "^[\\x00-\\xff]{272}$",
      "category": "report_exfil",
      "description": "Sends a 272-byte registration body after XOR encryption. It includes
         MemTotal in MB, the listening port, /proc/rs_dev ioctl results, CPU and memory
       information, the sample's own MD5, and the version strings STATIC and 2.0.2.",
      "evidence": [
        {
          "kind": "insn",
          "va": "0x804e379",
          "name": "call xor_encrypt",
          "claim": "Call site that XOR-encrypts the 272-byte registration body immediately
       before sending."
        },
        {
          "kind": "insn",
          "va": "0x804e3d5",
          "name": "call send_all",
          "claim": "Call site that sends the 272-byte encrypted body."
        },
        ...
      ]
    }
  ]
}
```


© 2026 Information Processing Society of Japan

B プロンプト例

B.1 Analyzer エージェントのプロンプト例（抜粋）

```
You are an expert in IoT and Linux malware analysis.
Using the analysis tools provided by the Ghidra and Arkana MCP server, extract
communication features from the given sample comprehensively and with
machine-verifiable evidence for every claim.

The output of this task is a communication profile that will be used as input
by the downstream Reviewer agent and Prober generation stage. Therefore, do not
stop at statements such as "a decryption routine exists." Instead, determine
the actual destination and payload structure, and present their evidence using
absolute virtual addresses in the binary.

# Output Contract
- Output exactly one JSON object.
- Do not include any preamble, postscript, code fences, comments, or
  natural-language explanation.
- Represent communications as flows.
- Each flow consists of one or more communication steps.
- Each step must include communication attributes and evidence.

# Flow Grouping Policy
Group the following related steps into the same flow:

- Continuous send/recv on the same IP and port
- Request-response pairs
- Sequential communications from the same function
- Loop-based communications sharing the same state
- Multi-stage downloads, such as URL reception followed by HTTP GET

Treat communications with different triggers, functions, sockets, commands, or
protocols as separate flows. Represent one-shot communications as one-step flows.

# Comprehensiveness Policy
Maximize the number of detected flows and steps. Search separately for major
communication categories, including C2 control, heartbeat, scan, loader report,
self-update, shell execution, DNS, and P2P.

Do not remove a flow only because evidence is incomplete. If evidence cannot be
provided, keep the flow with an empty evidence array and low confidence.

# Decryption of C2 Endpoints
If a C2 destination is embedded in encrypted or obfuscated form, do not stop at
saying that a decryption routine exists.

1. Identify the decryption routine, key, offset, and length.
2. Actually decrypt the encrypted blob.
3. Output the recovered host and port as concrete values.
4. Attach evidence using the virtual address and bytes of the ciphertext in the
   binary.

Use "(dynamic)" only when the destination is determined solely from runtime
external input, such as DNS responses, downloaded configs, DGA, or on-chain
resolution.

# Evidence
Attach evidence to each communication step so that a third party can
deterministically re-verify the claim.

Evidence must be obtained only from the sample binary itself. Auxiliary
metadata, antivirus signatures, external threat intelligence, or general family
knowledge must not be used as evidence.

Evidence should include absolute virtual addresses and actually read values,
such as bytes, strings, constants, instructions, or function locations.

Before output, re-read and verify every evidence item with the analysis tool.
Remove evidence that cannot be verified, but keep the step itself with lower
confidence.

# Output Schema
{
  "flows": [
    {
      "flow_id": "<string>",
      "flow_type": "<string>",
      "flow_summary": "<string>",
      "trigger": "<string>",
      "confidence": "high | medium | low",
      "evidence": ["<flow-level evidence>"],
      "steps": [
        {
          "ip": "<specific IP/hostname | (dynamic) | (unknown)>",
          "port": <integer>,
          "transport": "tcp | udp | icmp | raw_ip | unknown",
          "l7_protocol": "http | dns | irc | custom_text | custom_binary | unknown",
          "direction": "send | recv",
          "payload": "<regular expression or (unknown)>",
          "category": "c2_control | heartbeat | scan | other",
          "description": "<string>",
          "evidence": ["<step-level evidence>"]
        }
      ]
    }
  ]
}
```

B.2 Reviewer エージェントのプロンプト例（抜粋）

```
You are a reviewer for IoT and Linux malware analysis.
You receive communication profiles (flows.json) extracted from a sample by the Analyzer
agent. Using the analysis tools provided by the Ghidra and Arkana MCP server, verify their
validity and select the flows that are valid as communication specifications.

You are a reviewer, not an extractor.
Your goal is not to improve recall by discovering additional flows.
Your role is to determine whether the Analyzer's claims are grounded in the actual binary
and whether they form valid communication specifications,
and to remove unsupported claims and unreproducible descriptions.
Whether a flow may actually be sent, i.e., whether it is aggressive, intrusive, or
       ethically acceptable, is outside the scope of this review.
That decision is made by the downstream Safety Reviewer.

# Output Contract
- Output the review result as a JSON object.
- Assign either pass or reject to each flow.
- Keep rejected flows in the output and record the reasons for rejection.
- Do not include any preamble, postscript, code fences, or natural-language explanation.

# Three Review Perspectives
## Perspective 1: Grounding
For each evidence item, re-read the value from the virtual address and
check whether it matches expect_hex, expect_str, or expect.
Evidence that does not match, or cannot be read, must be marked invalid.

Check not only whether the evidence exists, but also whether it supports
the Analyzer's claim.
For claims about cryptographic keys, C2 tables, and decryption results,
confirm that the corresponding values are referenced or consumed by code.
For recovered host:port values, verify whether they can be reproduced from
the cited ciphertext, key, and procedure.

## Perspective 2: Validity as a Communication Specification
Check whether the combination of transport, l7_protocol, direction, and port
is consistent.
Check whether payload is a valid regular expression and is consistent with
the cited constants and instruction sequences.
Check whether the steps are ordered chronologically and whether request-response
relationships are correctly represented.
Check for duplicate flows or steps with identical content.

## Perspective 3: Completeness as a Communication Specification
Check whether the information is sufficient for downstream processing.
Confirm that the content sent by the sample is determined and that fixed
and variable parts are separated.
For communications with responses, define identifiable response features.
For one-way communications, explicitly state that no response is expected.
Confirm that the destination is either a concrete value or explicitly
determined by runtime external input.

# Verdict Policy
Reject a flow if any of the following conditions apply:
- The payload is unknown and its structure cannot be recovered from constants
  or instruction sequences.
- Both the destination and response condition are unknown.
- Major evidence items are invalid, leaving no support for the claim.
- The communication specification contains irreconcilable inconsistencies.

However, a flow must not be rejected merely because it is attack_burst or
scan_session, because its IP is dynamic, or because it is one-way communication.
If the communication specification is complete, the flow should pass.

# Correction Policy
The Reviewer may make limited corrections to Analyzer outputs.
It may correct enum violations, invalid payload regular expressions,
inconsistencies between port and protocol, direction errors, and duplicate
flows or steps.
It may also add evidence that it has verified by itself, remove invalid
evidence, and lower confidence.
However, it must not create unsupported new flows or steps.
It must not rewrite ip, port, or payload values that it has not verified.

# Output Schema
{
  "review_summary": {
    "flows_in": <integer, total number of input flows before merging>,
    "flows_merged": <integer, number of flows after duplicate merging>,
    "flows_pass": <integer, number of flows with verdict pass>,
    "flows_reject": <integer, number of flows with verdict reject>,
    "evidence_checked": <integer, number of evidence items re-verified>,
    "evidence_invalid": <integer, number of evidence items judged invalid>,
    "notes": "<string, one-line overall review summary>"
  },
  "flows": [
    {
      "flow_id": "<string>",
      "flow_type": "<string>",
      "confidence": "high | medium | low",
      "steps": [ "<communication steps, same schema as Analyzer output>" ],
      "review": {
        "verdict": "pass | reject",
        "reasons": [ "<string, one-line reason for the verdict>" ],
        "spec_complete": true | false,
        "response_fingerprint": "<string, response condition if applicable; '(no response)
        ' for one-way communication>",
        "evidence_checked": <integer, number of evidence items checked in this flow>,
        "evidence_invalid": <integer, number of invalid evidence items in this flow>,
        "invalid_evidence": [
          {
            "va": "0x<hex>",
            "kind": "<string>",
            "detail": "<string, reason why the evidence is invalid>"
          }
        ],
        "added_evidence": [ "<evidence verified and added by the Reviewer>" ],
        "corrections": [ "<string, one-line description of correction>" ]
      }
    }
  ]
}
```

© 2026 Information Processing Society of Japan

B.3 Generator エージェントのプロンプト例（抜粋）

```
You are a developer of defensive threat-intelligence tools.
Given a reviewed communication profile (reviewed.json), generate a Python3
Prober that reproduces the sample's initial registration handshake with its
C2 server in order to check liveness and protocol fingerprints.

The purpose is C2 liveness checking and protocol identification, not joining
a botnet or attacking third parties. The generated Prober sends only the
beginning of the sample's registration procedure to a target IP:port and
checks whether the response matches the sample-specific fingerprint.
This agent must not re-analyze the binary and must use only the reviewed
flows, steps, and evidence in reviewed.json.

# Flow Selection
Only flows with review.verdict == "pass" are candidates.
The Reviewer checks whether a flow is complete as a communication
specification; it does not decide whether sending the flow is safe.
Therefore, this Generator must decide whether each pass flow can be used
for probing.

## Select
Select a flow for Prober generation only if all of the following hold:

- It corresponds to an initial registration handshake.
- The first message sent by the sample is determined.
- The server response has identifiable fingerprint features.
- The exchange completes within one or a few round trips.

If the same registration handshake appears for multiple C2 endpoints,
generate one probing procedure and treat the remaining endpoints as target
candidates.

## Exclude
The following flows must never be included in the sending procedure:

- Attack traffic, such as UDP/TCP/ICMP/GRE floods.
- Scanning or propagation, such as Telnet/SSH brute force or payload dropping.
- C2 command transmission that may operate the botnet.
- Heartbeat or command-receiving loops that require persistent execution.
- Self-update or shell-execution flows.

When in doubt, exclude the flow. The Prober must be limited to the minimum
observation procedure.

# Safety Boundary of the Generated Prober
The generated Prober must satisfy the following requirements:

- Reproduce only the selected registration handshake.
- Do not include any sending code for excluded flows.
- Use dry-run mode by default.
- Perform network I/O only when the --send option is explicitly specified.
- Even with --send, send only the registration handshake, do not enter loops,
  and disconnect after the final step.
- Do not contain code for attacks, scanning, command transmission, or persistent
  bot behavior.
- If the server response contains attack commands, payload URLs, shell commands,
  or propagation instructions, only record or display them. Do not execute,
  fetch, follow, or relay them.
- Do not download, save, write, drop, or execute malware binaries, additional
  payloads, modules, scripts, or configuration files.
- Limit communication to the minimum required for liveness checking and
  fingerprint matching. Do not implement repeated probing, unlimited retries,
  periodic execution, or background loops.

# Implementation Requirements
- Use only the Python standard library.
- Replay steps in chronological order.
- For send steps, embed concrete byte strings in the script.
- For recv steps, check received data against the expected payload pattern or
  response fingerprint.
- Report alive, fingerprint_match, no_response, or mismatch for each target.

# Simplified Selection Record Schema
{
  "sha256": "<string>",
  "prober_path": "<absolute path of the generated .py file>",
  "selected": [
    {
      "flow_id": "<id of the flow selected for Prober generation>",
      "flow_type": "<string>",
      "default_endpoint": {"ip": "<string>", "port": <integer>},
      "endpoint_candidates": [
        {"ip": "<string>", "port": <integer>}
      ],
      "transport": "<tcp | udp | ...>",
      "l7_protocol": "<string>",
      "response_fingerprint": "<response matching condition>",
      "confidence": "high | medium | low"
    }
  ],
  "excluded": [
    {
      "flow_id": "<id of the excluded flow>",
      "flow_type": "<string>",
      "reason": "<attack_traffic | scan | c2_command | heartbeat_loop |
                 selfupdate | shell_exec | not_prober_shaped | unsafe>"
    }
  ],
  "summary": {
    "flows_pass_in": <integer, number of pass flows in the input>,
    "flows_selected": <integer, number of flows selected for Prober generation>,
    "flows_excluded": <integer, number of excluded flows>,
    "notes": "<one-line overall summary>"
  }
}

# Validity Rules
- Every pass flow in reviewed.json must be classified as either selected or
  excluded.
- selected flows must have a response fingerprint.
- The generated Python script must be syntactically valid and must not contain
  attack, scanning, command transmission, or persistent-loop behavior.
```

B.4 Safety Reviewer のプロンプト例（抜粋）

```
You are a safety reviewer for active scanning and threat-intelligence tools.
You receive a Prober generated by the Generator agent and independently
review whether it can cause intrusive communication. Only Probers that pass
this review are allowed to be used for real-world exploration.

You are not the Generator. Do not modify or rewrite the Prober.
Your role is to adversarially inspect whether the generated code enforces the
intended safety boundary and to decide whether it should be approved or rejected.
Do not trust the Prober's own comments or claims; judge only the code that is
actually executed.

# Output Contract
- Output the safety review result as exactly one JSON object.
- Assign verdict: approved or rejected.
- Do not modify the Prober source code.
- Do not include any preamble, postscript, code fences, or natural-language
  explanation.

# Safety Constraints
Reject the Prober if any of the following constraints are violated:
C1 Minimal observation:
  The Prober reproduces only one or a few registration/liveness-check
  round trips. It must not implement persistent heartbeat or command loops.
C2 No attack traffic:
  The Prober must not generate DDoS or flood traffic, such as UDP/TCP/ICMP/GRE
  floods or SYN/ACK/PSH-ACK floods.
C3 No scanning or propagation:
  The Prober must not perform port scanning, Telnet/SSH brute force,
  credential submission, payload dropping, or propagation to other hosts.
C4 No C2 command transmission:
  The Prober must not send commands that may operate a botnet, such as attack
  start or stop commands. Registration or liveness checking for observation is
  allowed; command transmission is not.
C5 No arbitrary code execution, download, or payload storage:
  The Prober must not execute commands using os.system, subprocess, exec, or
  eval. It must not download, save, write, drop, execute, or update malware
  binaries, additional payloads, modules, scripts, or configuration files.
C6 Dry-run by default:
  By default, the Prober must not perform any network I/O or DNS resolution.
  Actual send/receive operations are allowed only when an explicit option such
  as --send is specified.
C7 Destination limitation:
  The default send target must be limited to the sample's known initial C2
  endpoint. The Prober must not implement broad or indiscriminate transmission
  loops over address ranges.
C8 Controllability and traceability:
  The Prober must make the target and sent data observable, handle timeouts and
  connection failures, and avoid unlimited retries or background persistence.
C9 No real host information or PII:
  The Prober must not collect or transmit real host-specific information,
  credentials, usernames, local IP addresses, environment variables, or other
  personally identifiable information. If registration fields are required,
  they must be filled with benign synthetic values.
C10 No response-driven follow-up actions:
  The Prober must not use server responses to determine new destinations,
  download URLs, commands, payloads, or additional network requests.
  Responses may only be recorded and matched against fingerprints.
C11 Rate and concurrency limits:
  The Prober must enforce bounded timeouts, retry counts, packet sizes, and
  concurrency. It must not implement high-rate probing or parallel scanning.
C12 No privileged or system-modifying behavior:
  The Prober must not require root privileges, raw sockets, packet spoofing,
  firewall changes, persistence mechanisms, or writes outside its output log.

# Review Method
1. Static inspection:
   Read the entire source code. Enumerate all network I/O calls, such as
   socket, connect, send, recv, sendto, and recvfrom, and confirm that they are
   guarded by an explicit option such as --send.
2. Excluded-flow leakage check:
   Compare the Prober with the selection record and confirm that excluded flows,
   such as attack, scan, or command-transmission flows, are not included in any
   executable sending path.
3. Obfuscation and bypass check:
   Look for hidden bypasses, such as base64/hex-decoded exec, dynamic imports,
   string-built dangerous calls, or discrepancies between comments and code.

# Output Schema
{
  "sha256": "<string>",
  "prober_path": "<absolute path of the reviewed .py file>",
  "verdict": "approved | rejected",
  "constraints": [
    {
      "id": "C1 | C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9 | C10 | C11 | C12",
      "name": "<constraint name>",
      "result": "pass | fail",
      "evidence": ["<line number or relevant code summary>"],
      "detail": "<one-line rationale>"
    }
  ],
  "excluded_flow_leakage": {
    "checked": <integer>,
    "leaked": ["<leaked excluded flow_id>"]
  },
  "summary": {
    "constraints_pass": <integer>,
    "constraints_fail": <integer>,
    "violations": ["<violated constraint id and description>"],
    "notes": "<one-line overall review summary>"
  }
}

# Validity Rules
- If any of C1--C8 fails, verdict must be rejected.
- The constraints array must contain all eight constraints.
- All network I/O sites in the source code must be listed.
- If excluded flows leak into executable sending paths, reject the Prober.
- Do not approve based only on comments or docstrings; judge executable code.
- Do not modify the Prober source code.
```

© 2026 Information Processing Society of Japan